In-Line Pharmaceutical Tablet Quality Control using Computer Vision

Mahnoor Zia
Dept. of Computer and Information Systems Engineering
NED University of Engineering and Technology
Karachi, Pakistan
mahnoor.zia205@gmail.com

Laiba Iqrar
Dept. of Computer and Information Systems Engineering
NED University of Engineering and Technology
Karachi, Pakistan
laibaiqrarahmedkhan@gmail.com

Anoosha Khalid
Dept. of Computer and Information Systems Engineering
NED University of Engineering and Technology
Karachi, Pakistan
anooshakhalid999@gmail.com

Manahil Ejaz
Dept. of Computer and Information Systems Engineering
NED University of Engineering and Technology
Karachi, Pakistan
manahilejaz16@gmail.com

Abstract

Quality control of traditional medicine tablet relies mostly on off-line testing done on a restricted number of samples, hence inspection is either time-consuming, labor-intensive, or prone to omitting important flaws. This restriction stresses how important an automated inline inspection method capable of guaranteeing consistent product quality while reducing waste is. For real-time inspection of pharmaceutical tablet cores, this study offers an inline tablet quality control approach combining deep learning with traditional machine vision. First looked at were three classical mathematical vision variations using combinations of Gaussian smoothing, HSV colour-space analysis, Otsu thresholding, morphological operations, convex-hull subtraction, Laplacian edge analysis, spatial zoning, and colourimetric filtering for defect location and interpretation. Based on these results, a hybrid vision architecture and deep-learning segmentation pipeline were created to increase robustness and speed of deployment. Categories of surface faults including normal tablets, scratches, edge chipping, and capping were noted and utilized for model training and assessment. Emphasis on accuracy, low latency, and industrial throughput enabled training and optimization of a YOLO-family segmentation model for real-time inference. Experimental data showed that the best-performing YOLO11n-seg setup obtained a mIoU of 0.7582 and a mAP@0.5 of 96.28%. With an inference latency of 19.29 ms, the system obtained a throughput of 42.9 Frames Per Second (FPS), therefore surpassing traditional two-stage frameworks such as Mask R-CNN, which showed a latency of 224.50 ms and an operational frame rate of 4.4 FPS. To show the viability of a low-cost and widely implementable tablet inspection tool, the whole system was also deployed on Raspberry Pi 4 with real-time monitoring via a Grafana dashboard.

INTRODUCTION

Pharmaceutical tablet manufacturing demands rigorous quality control to guarantee product safety, efficacy, and regulatory compliance. During production, tablets can develop surface defects such as edge chipping, capping, scratches, spots, and smear marks. Traditionally, quality inspection is performed manually on a small random sample of each batch. This approach is labor‑intensive, subjective, and unable to keep pace with modern high‑speed production lines that produce thousands of tablets per hour. As a result, defective tablets may escape detection, posing risks to patients and leading to costly product recalls.

Automated visual inspection (AVI) systems have been introduced to overcome the limitations of manual inspection. However, commercial AVI solutions are typically expensive, rely on proprietary "black‑box" algorithms, and are difficult to retrain or adapt to new products. They also tend to produce only a pass/fail decision or coarse bounding‑box localisation, lacking the pixel‑level defect characterisation that is essential for root‑cause analysis and quantitative quality evaluation. These barriers leave small and medium‑sized pharmaceutical manufacturers without an accessible, high‑performance inline inspection solution.

Recent advances in computer vision and deep learning offer a promising pathway toward low‑cost, adaptable inspection systems. End‑to‑end deep learning models can learn complex defect patterns directly from annotated data. However, they require large datasets, substantial computational resources, and often struggle with industrial variability such as changing lighting, tablet orientation, and background clutter. Conversely, classical computer vision techniques—based on morphological operations, colour‑space analysis, and geometric measurements—are fast, interpretable, and can run efficiently on edge hardware. Yet, purely rule-based methods are fragile; they fail when faced with the unpredictable appearance of real-world defects and require hand-tuning for every new tablet type.

To address these challenges, this research proposes a hybrid computer vision framework for real-time, inline pharmaceutical tablet defect inspection. The framework combines the speed and interpretability of classical mathematical vision with the defect-recognition power of deep learning segmentation, and it deploys the entire pipeline on low-cost edge hardware. The system is built around a custom conveyor-based imaging platform with controlled illumination that captures high-resolution images of moving tablets. A three-stage methodological investigation is conducted:

1. ***Classical Computer Vision Approaches*** – Three distinct mathematical pipelines are developed and tested on live conveyor video. These pipelines use Gaussian smoothing, HSV colour-space conversion, Otsu's adaptive thresholding, morphological refinement, convex-hull subtraction, Laplacian gradient analysis, region-constrained spatial zoning, colourimetric profiling, CLAHE contrast enhancement, Black-Hat morphological transforms, and geometric aspect-ratio filtering. Each variation explores a different trade-off between speed, robustness, and defect isolation accuracy.

2. ***Deep Learning Segmentation Models*** – A custom dataset of 606 images is created with hierarchical polygon-based instance segmentation annotations (tablet-level and defect-level masks). Using this dataset, YOLO11n-seg and Mask R-CNN architectures are trained under multiple optimisation and augmentation strategies. The top-performing YOLO11n-seg model achieves 96.28 % mAP@0.5, mIoU of 0.7582, and an inference latency of 19.29 ms (42.9 FPS), while Mask R-CNN yields superior segmentation precision (mIoU 0.9676) at the cost of much slower inference (224.5 ms).

3. ***Hybrid Classical-Deep Learning Pipeline*** – A decoupled architecture is designed in which classical computer vision performs robust tablet localisation, geometric alignment, and background suppression, and a lightweight deep learning model processes only the isolated tablet region for defect segmentation. This hybrid approach reduces computational complexity, accelerates inference, and eliminates the need for the deep learning model to learn spatial invariance.

The complete system is integrated into a real-time inference pipeline with a web-based dashboard (Grafana) for live inspection statistics, and the final model is deployed on a Raspberry Pi 4, proving the feasibility of a low-cost, inline, 100 % inspection system.

The principal contributions of this work are:

- ***A multi-domain annotated dataset*** with hierarchical polygon masks, combining controlled laboratory images and operational conveyor frames to bridge the domain gap.

- ***A thorough evaluation of classical mathematical vision methods*** for tablet defect detection, highlighting their strengths and industrial limitations.

- ***A comprehensive comparison of deep learning segmentation architectures*** (YOLO11n-seg and Mask R-CNN) under varied training and augmentation strategies, with a focus on the accuracy-efficiency trade-off.

- ***A novel hybrid inspection pipeline*** that decouples tablet localisation (classical) from defect segmentation (deep learning), achieving a practical balance between speed and precision.

- ***End-to-end deployment on edge hardware*** with a live operator dashboard, demonstrating a complete, scalable solution for pharmaceutical inline quality control.

Following are some types of classes that we worked on:

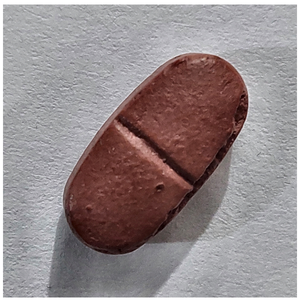

Fig. 1. Capped Defect

Fig. 2. Chipped Defect

Fig. 3. Normal

2. Literature Review

2.1. Related Work

Automated visual inspection of pharmaceutical tablets has evolved from simple rule-based algorithms to sophisticated deep-learning pipelines. Early inspection systems relied on classical computer vision techniques such as colour thresholding, morphological operations, and geometric feature extraction. [1] proposed a method for detecting missing, broken, or colour-mismatched tablets in blister packs using golden template matching and colour segmentation. While effective for fixed, controlled setups, such approaches are extremely sensitive to lighting changes, tablet orientation, and background clutter. They also require manual parameter tuning for every new product variant, making them unsuitable for dynamic inline production environments.

Beyond template-based inspection systems, several studies explored more advanced classical computer vision techniques for pharmaceutical quality control and industrial surface inspection. These approaches aimed to improve robustness while maintaining low computational requirements. Traditional machine-vision pipelines typically consisted of image acquisition, preprocessing, segmentation, feature extraction, and rule-based classification [7], [8]. Noise-reduction techniques such as Gaussian filtering and median filtering were commonly applied to suppress sensor artefacts while preserving defect boundaries, whereas adaptive thresholding methods including Otsu thresholding and local thresholding were used to isolate objects under varying illumination conditions [9]. Morphological operations including erosion, dilation, opening, and closing further improved segmentation quality by removing noise and refining object contours [10].

Shape-based analysis became another widely adopted strategy for defect inspection. Contour extraction, connected-component analysis, convex hull estimation, and geometric descriptors such as circularity, compactness, eccentricity, and aspect ratio were employed to identify chipped edges, broken tablets, and dimensional inconsistencies [11]. Edge-based operators including Sobel, Canny, and Laplacian filtering enabled localisation of scratches, cracks, and abrupt surface discontinuities by emphasising high-frequency intensity transitions [12]. Additionally, colour-space transformations from RGB into HSV and LAB improved robustness against illumination variation and enhanced detection of coating defects and exposed tablet cores [13].

Although classical vision methods remain attractive because of their interpretability, deterministic behaviour, and low computational overhead, they depend heavily on manually selected thresholds and handcrafted features [14]. Their performance typically degrades under changing illumination, background clutter, orientation variation, and unseen defect morphology, limiting their ability to generalise across industrial production environments [15].

The Shift to Deep Learning brought significant improvements in defect detection accuracy and robustness. Convolutional neural networks (CNNs) can automatically learn discriminative features from annotated images, eliminating the need for hand-engineered rules. [2] used a VGG-16 classifier to detect chipping, breaking, colour non-uniformity, and speckling on film-coated tablets with 99.7% accuracy. However, their method relies on a 3D-printed tray to isolate individual tablets and only outputs a class label, providing no spatial information about the defect location or extent. [3] demonstrated the use of CNNs for internal tablet defect detection, but again, the task was limited to classification rather than precise localisation.

Object Detection Models Such as YOLO and Faster R-CNN have been applied to simultaneously locate and classify defects. [4] combined YOLOv5 with classical image analysis to measure coating thickness and recognise surface defects on film-coated tablets in real time. Their system achieved high throughput but was constrained to bounding-box detection, which cannot provide the pixel-level defect boundary required for quantitative quality evaluation. Similarly, [5] proposed an enhanced YOLOv8 model with coordinate attention and a bi-directional feature pyramid network for blister pack defect detection, achieving 97.4% mAP@0.5 at 79FPS. Although extremely fast, this work did not address segmentation or edge-device deployment. instance segmentation

The need for exact defect shape measurement has driven the adoption of instance segmentation architectures. Diószegi et al. [6] applied deep learning segmentation to tablet surface images for simultaneous defect detection and prediction of disintegration time and crushing strength. Their work illustrates that pixel-level masks enable the extraction of quantitative quality indicators that go beyond simple pass/fail decisions. However, the study was conducted on static, high-quality images and did not consider the challenges of real-time conveyor-belt inspection, domain shift, or edge deployment.

2.2. Research gap:

The existing body of work demonstrates that classical methods are too brittle, detection models lack spatial precision, and segmentation studies rarely address the full industrial pipeline—including hardware integration, real-time operation on low-cost edge devices, and operator-friendly dashboards. Moreover, few works combine the interpretability and computational efficiency of classical vision with the accuracy of deep learning segmentation in a decoupled, deployable framework.

This paper addresses these gaps by proposing a hybrid inspection pipeline that:

1. uses classical computer vision for robust tablet localisation and background suppression

2. employs YOLO11n-seg for pixel-accurate defect segmentation

3. evaluates both classical and deep-learning approaches on a custom dataset that includes real conveyor-belt frames

4. deploys the final system on a Raspberry Pi 4 with a live Grafana dashboard.

Thus, our contribution is not merely a model comparison, but a complete, industrially-relevant inspection framework that bridges the gap between high-accuracy research models and practical, low-cost inline quality control.

3. Review of Related Concepts

This section provides a brief overview of the core techniques that the proposed inspection system builds upon. The reader is assumed to be familiar with basic image processing and deep learning; here we only highlight the specific methods adopted in our pipeline.

3.1. Image Preprocessing for Industrial Environments

Industrial tablet images often suffer from sensor noise, uneven illumination, and motion blur. To create a stable input for both classical and deep-learning pipelines, we apply three standard preprocessing steps:

- ***Noise reduction:*** A bilateral filter is used because it smooths flat regions while preserving sharp edges vital for detecting fine cracks and chips.
- ***Lighting correction:*** CLAHE (Contrast Limited Adaptive Histogram Equalization) is applied to the lightness channel in the LAB colour space. This method enhances local contrast without amplifying noise or distorting colour, effectively correcting shadows and uneven illumination.
- ***Colour space conversion:*** RGB images are transformed to HSV. The separation of hue, saturation, and value allows colour-based defect detection to be independent of brightness.

These steps are simple, non-destructive, and run in real time on the CPU, making them suitable for edge deployment.

3.2. Classical Machine Vision Techniques

Classical computer vision relies on deterministic operations rather than learned models. The following are the key operations used in our mathematical pipelines:

- ***Thresholding and segmentation:*** Otsu's method automatically finds an optimal intensity threshold to separate the bright tablet from the dark conveyor background. It does not require any training or manual tuning.
- ***Morphological operations:*** Opening (erosion followed by dilation) removes small noise; closing (dilation followed by erosion) fills holes inside the tablet mask. These operations clean up the binary mask of the tablet.
- ***Contour analysis and convex hull:*** The largest external contour is extracted to define the tablet outline. The convex hull of that contour is computed; chipping defects are found by subtracting the tablet mask from its convex hull. Any leftover region indicates an edge break.
- ***Laplacian edge detection:*** The Laplacian operator highlights rapid intensity changes, such as crack lines. By summing the Laplacian response inside the tablet area, we detect capping defects.
- ***HSV colour profiling:*** A chipped area often exposes the uncoated core, which has low saturation and high brightness. A simple logical condition ($S < 60$ and $V > 170$) isolates such regions.
- ***Black-hat transform:*** This morphological operation extracts dark, thin structures (like cracks) from a varying background.

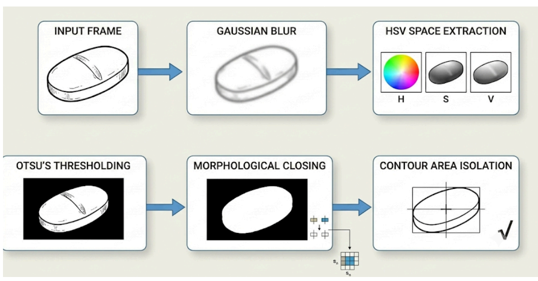

Figure 4. Tablet localization pipeline

3.3. Deep Learning for Instance Segmentation

Deep learning models learn to recognise defects directly from annotated data. Unlike classical methods, they generalise to new variations without manual rule changes.

- ***Convolutional Neural Networks (CNNs):*** CNNs extract hierarchical features (edges → textures → object parts) and are the backbone of modern vision systems.
- ***Object Detection vs. Segmentation*:** Detection outputs a bounding box around a defect; segmentation outputs a pixel-level mask. Only segmentation gives the exact shape, size, and position needed for pharmaceutical quality assessment.
- ***YOLO-seg:*** YOLO (You Only Look Once) is a single-stage detector that achieves real-time speed. Its segmentation variant, YOLOv11n-seg, adds a mask head that predicts instance masks directly. The nano version contains only 2.9 M parameters, making it ideal for edge devices.
- ***Mask R-CNN:*** A two-stage model that first proposes regions of interest and then refines boxes and predicts masks. It achieves higher mask accuracy than YOLO but is much slower (~200 ms vs. ~20 ms per frame).

3.4. Evaluation Metrics

We use the following standard metrics to quantify performance:

- ***Precision and Recall:*** Precision is the fraction of detected defects that are true defects; recall is the fraction of real defects that are detected. The F1-score balances both.
- ***Intersection over Union (IoU):*** The overlap between the predicted mask and the ground-truth mask, divided by their union. It measures mask accuracy.
- ***Mean Average Precision (mAP):*** The area under the precision-recall curve, averaged over all classes. We report mAP@0.5 (IoU threshold 0.5) for detection quality and mAP@0.5:0.95 for strict mask accuracy.
- ***Inference Latency and FPS:*** The time (in milliseconds) to process one frame, and the number of frames processed per second. These directly determine whether the system can keep up with a production line.

These metrics allow fair comparison between YOLO, Mask R-CNN, and the classical pipelines.

4. Description of Methodology

This section presents the complete workflow of the proposed inline tablet inspection system, from hardware-based image acquisition to the final inference pipeline. The methodology is structured into three major pillars: (1) the physical imaging setup, (2) the dataset creation and annotation protocol, and (3) the computer vision and deep learning models that perform defect detection and segmentation.

4.1 Hardware Setup and Image Acquisition

The physical inspection platform, shown in **Figure 5.** , consists of a conveyor belt, a high-resolution RGB camera, controlled LED illumination, and a single-board computer that handles inference.

- ***Conveyor mechanism:*** Tablets are transported on a continuous belt at an adjustable speed that ensures each tablet remains within the imaging zone long enough to be captured sharply.
- ***Camera:*** A 1080p auto-focus USB camera is mounted vertically above the belt, capturing a top-down view of the tablet surfaces.
- ***Illumination:*** A ring light combined with a diffuser provides uniform, shadow-free lighting. The diffuser minimises specular reflections, which are particularly important for glossy tablet coatings.
- ***Edge device:*** A Raspberry Pi 4 (8 GB RAM) runs the inference pipeline in real time. For heavier training and evaluation, a cloud-based Tesla T4 GPU was used.

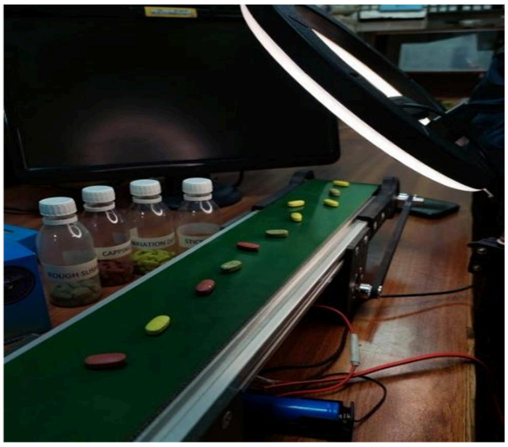

Figure 5. Photograph of the conveyor-based inspection hardware, showing the camera.

All camera parameters (exposure, focus, white balance) are fixed during acquisition to maintain image consistency. The belt speed and camera frame rate are balanced to prevent motion blur: with a 30 fps capture rate and a belt velocity of approximately 3.7 inches per second, each tablet appears in several consecutive frames without visible smearing.

4.2 Dataset Creation and Preprocessing

4.2.1 Data Sources

The image dataset was assembled from three sources to cover both laboratory-grade and production-line conditions:

1. **Controlled static images (545 frames):** Individual tablets were photographed under consistent lighting and orientation, yielding high-quality, clean images. Filenames follow a structured naming convention
2. **Conveyor video frames (263 frames):** Real production-line footage was recorded at 1920×1080 px and 30 fps. Frames were extracted at a stride of 10 (one frame every 0.33 s) to avoid redundant near-duplicates, then manually screened to remove heavily occluded or incomplete tablets.
3. **External supplementary images (34 frames):** A small set of additional tablet images obtained from an external pipeline was included to slightly increase diversity.

All raw frames were stored without initial filtering. The complete dataset after cleaning contains 606 image-label pairs.

4.2.2 Preprocessing

Two preprocessing operations are applied to every image before annotation or inference:

- ***Bilateral filtering:*** A spatial kernel that reduces sensor noise while preserving edge sharpness. This is critical because many defects (chips, cracks) are defined by precise boundaries.
- ***CLAHE (Contrast Limited Adaptive Histogram Equalization):*** Applied to the L-channel in the LAB colour space, CLAHE enhances local contrast and corrects uneven illumination without shifting colours.

Images are not resized during preprocessing; the original resolution is kept to preserve fine defect detail. For model training, online resizing to the target input size (640 px or 1280 px) is performed by the YOLO dataloader.

4.3 Annotation Protocol

A hierarchical two-level instance segmentation strategy was designed to decouple the object (whole tablet) from its surface anomalies. All annotations were performed in CVAT using polygon masks.

- ***Level 1 – Tablet mask:*** Every visible tablet is outlined with a polygon. This mask defines the region of interest and is labelled with one of four classes: `tablet`, `normal`, `chip`, or `cap`. (Note: `normal` tablets are those without any defects; `tablet` class is used for tablets where the image also contains a defect annotation elsewhere in the frame, preserving the tablet's identity.)
- ***Level 2 – Defect mask:*** Within each tablet polygon, any surface defect (chip or cap) is annotated as a separate polygon instance. These defect masks are spatially constrained inside the parent tablet mask. Normal tablets receive no defect masks.

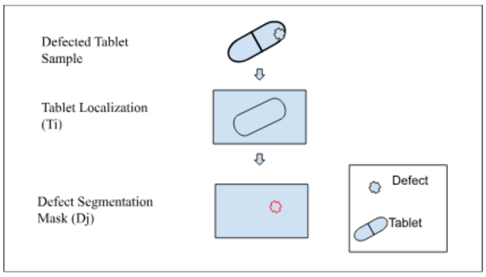

Figure 6. Illustration of the two-tier annotation concept

To speed up the tablet-mask generation, the Segment Anything Model 3 (SAM 3) was first applied to propose initial tablet boundaries; these were then manually refined. Defect masks, due to their small scale and subtle appearance, were annotated entirely by hand to ensure pixel-level accuracy.

4.4 Dataset Engineering and Splitting

4.4.1 Data Cleaning

A Python script was written to enforce strict 1:1 correspondence between image files and label files. The script recursively scans all sub-directories and removes any image that lacks a matching annotation file. After cleaning, 606 fully matched pairs remained.

4.4.2 Stratified Splitting

The 545 high-quality static images were split in a stratified 70/20/10 ratio into **training**, **validation**, and **test** sets, preserving class proportions. The 263 conveyor-belt frames were initially placed entirely in the training set to avoid noisy annotations contaminating the evaluation splits. In a later refinement, 44 conveyor frames were moved to the test set and duplicated into the validation set, so that the evaluation metrics would reflect real production conditions.

The final dataset partition is given in **Table 4.1**.

Partition	Images	Composition
Train	565	70% stratified static + all remaining conveyor frames + external images
Test	162	20% stratified static + copies of the 44 conveyor test frames
Valid	81	10% stratified static + 44 unique conveyor frames

Table 4.1. Dataset partition

4.4.3 Class Distribution

The number of individual instances per class in the training set is shown in **Table 4.2**. Because conveyor frames contain 4–7 tablets each, the total instance count is higher than the image count.

Class	ID	Instances
Tablet	0	484
Chip	1	261
Cap	2	245
Normal	3	270

Table 4.2. Training-Set Instance Distribution

The chip and cap classes are slightly under-represented, which is expected given the rarity of defects in real production. This mild imbalance is addressed by strong online augmentations during training.

4.5 Computer Vision and Deep Learning Pipelines

The defect inspection task was approached through three distinct methodological tracks, reflecting the evolution from purely rule-based to purely learned and finally to a hybrid system.

4.5.1 Classical Mathematical Vision Variations

Three algorithmic variants were implemented using only classical image processing techniques. All share a common front-end:

1. ***Preprocessing:*** Gaussian blur → HSV conversion → Otsu thresholding on the Value channel → morphological closing.
2. ***Tablet localisation:*** The largest external contour is extracted as the tablet region.
3. ***Defect detection:*** Each variant applies a different defect-specific logic:

- ***Variation 1 – Global geometry:*** Chipping is detected by subtracting the tablet mask from its convex hull; capping is detected by summing the Laplacian response over the tablet area.
- ***Variation 2 – Spatial zoning:*** Chipping is restricted to a boundary ring (erosion-based), and capping is obtained by binarising the Laplacian map and applying morphological filtering.
- ***Variation 3 – Colourmetric & morphological filtering:*** Chipping is isolated through HSV saturation-brightness profiling (exposed white core), while capping is detected by a Black-Hat transform followed by aspect-ratio filtering of contours.

These variations are described in detail with pseudocode and example outputs in the supplementary material. Their real-time performance is summarised in Section 5.

4.5.2 Deep Learning Instance Segmentation

Two families of deep neural networks were trained and compared.

YOLO11n-seg (Single-Stage)

YOLO11n-seg is a lightweight segmentation model with 2.9 million parameters. Training was conducted in a two-stage pipeline:

- ***Stage 1 – Tablet detector:*** A YOLO11n-seg model is trained on full-resolution (1280 px) conveyor frames to detect and localise all tablets. It outputs bounding boxes for each tablet, achieving a tablet recall of 0.944.
- ***Stage 2 – Defect segmenter:*** For each detected tablet, a crop with a 20 % margin is extracted. A second YOLO11n-seg model is trained on these crops to segment only the defect classes (chip and cap). The class IDs are remapped (chip→0, cap→1), and normal tablets (crops without defects) are discarded. This second model operates at 640 px input size.

Three training configurations were explored for Stage 2:

- ***Baseline***: moderate augmentation (HSV jitter, small mosaic), AdamW optimiser, lr=0.0008, batch 16.
- ***Augmented:*** heavy augmentation (mosaic=0.5, copy-paste=0.3, flips, scaling), SGD optimiser, batch 8.
- ***Regularized:*** partial backbone freezing, dropout 0.3, Cosine learning rate decay, AdamW, batch 4.

Model ID	mAP50	AP75	mIOU	Latency (ms)
1.2 Baseline	0.664	0.6423	0.428	**10.57**
1.1 Augmented	0.9628	0.8049	**0.7582**	19.29
1.3 Regularized	0.8795	0.7654	0.5989	13.65

Table 4.3. Comparative Analysis of Stage-2 Model Performance and Inference Latency

The best-performing configuration was the Augmented model, which achieved the highest mAP and mIoU.

Mask R-CNN (Two-Stage)

As a high-accuracy baseline, a Mask R-CNN with ResNet-50+FPN backbone was trained on the same dataset. Three optimisation strategies were tested:

- SGD baseline (lr=0.00025)
- AdamW with backbone frozen up to Stage 2
- SGD with Cosine annealing and warm-up

Model ID	mAP50	AP75	mIOU	Latency (ms)
Run1_SGD	**0.8536**	0.5894	0.9665	226.7
Run2_AdamW	0.845	0.569	0.9663	217.8
Run3_cosine	0.8407	0.5818	**0.9676**	**224.5**

Table 4.4. Comparative Analysis of Mask RCNN Performance and Inference Latency

The best Mask R-CNN model (Run3_Cosine) served as the segmentation quality benchmark in the final comparison.

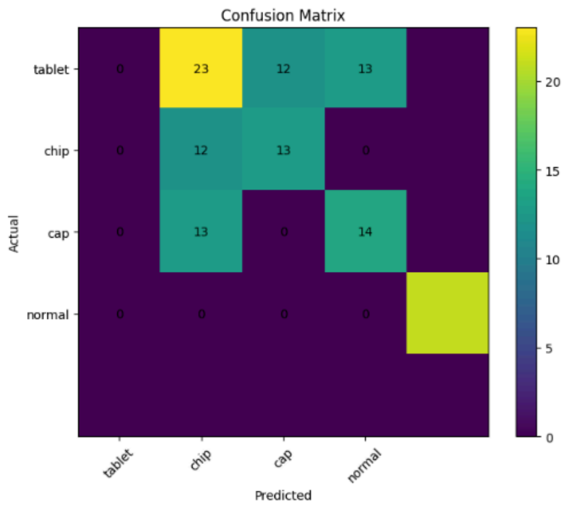

Figure 7. Confusion matrix of Run3_SGDCosine (Mask RCNN) best Model

4.5.3 Hybrid Classical-Deep Learning Approach

To combine the speed of classical methods with the precision of deep learning, a hybrid pipeline was designed:

1. ***Classical localisation:*** The tablet region is extracted using Otsu thresholding, morphological refinement, and contour analysis—exactly as in the mathematical vision pipeline.
2. ***Geometric alignment:*** The tablet image is rotated to a canonical orientation using second-order moments.
3. ***Deep learning segmentation:*** The oriented, cropped tablet is passed to a lightweight YOLO11n-seg model that segments only the defect regions.

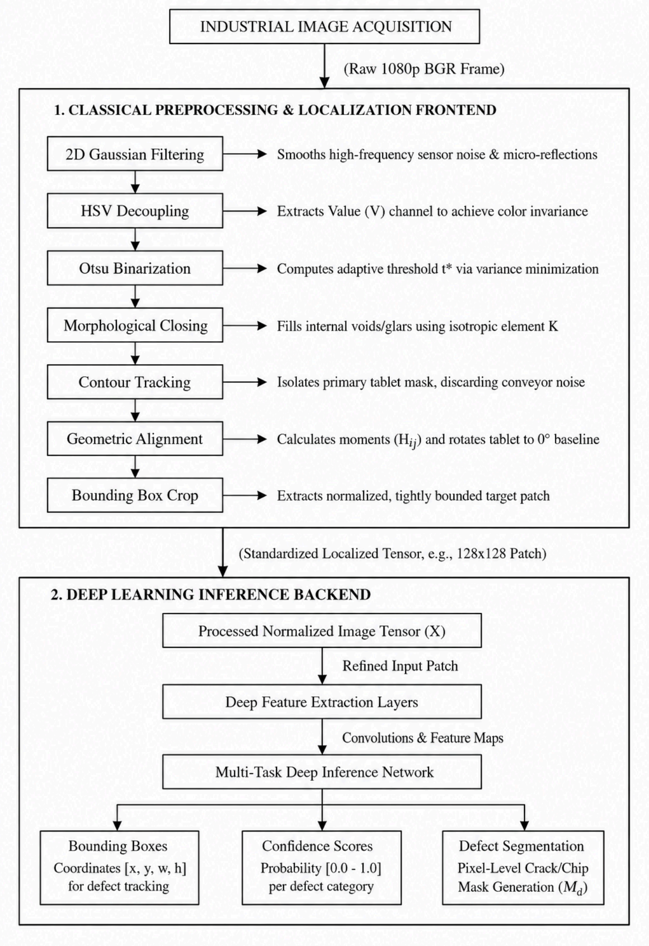

Figure 8. Mathematical and Deep Learning hybrid architecture

Because the deep learning model receives only an aligned, background-free tablet, it does not need to learn spatial invariance or background suppression. This dramatically reduces the required training data and speeds up inference, while retaining pixel-level accuracy.

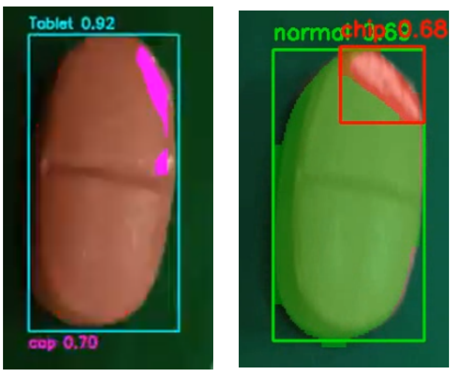

Figure 9. Real Time results of hybrid architecture on moving conveyor belt

4.6 Inference Pipeline and Edge Deployment

The final deployment pipeline runs on a Raspberry Pi 4. The workflow is:

1. ***Frame capture***: The USB camera provides a live feed.
2. ***Tablet detection (Stage 1 or classical):*** In the pure deep-learning pipeline, the 1280 px Stage 1 model detects tablets. In the hybrid pipeline, classical localisation is used.
3. ***Crop extraction:*** Each detected tablet is cropped with a 5 % margin.
4. ***Defect segmentation (Stage 2):*** The crop is processed at 640 px by the Stage 2 YOLO model or by the classical variation.
5. ***Visualisation and logging:*** Segmentation masks are overlaid on the original frame, and the detection results (class, confidence, mask) are logged to an InfluxDB time-series database.
6. ***Operator dashboard:*** Grafana displays live statistics, defect counts, pass/fail rates, and historical trends.

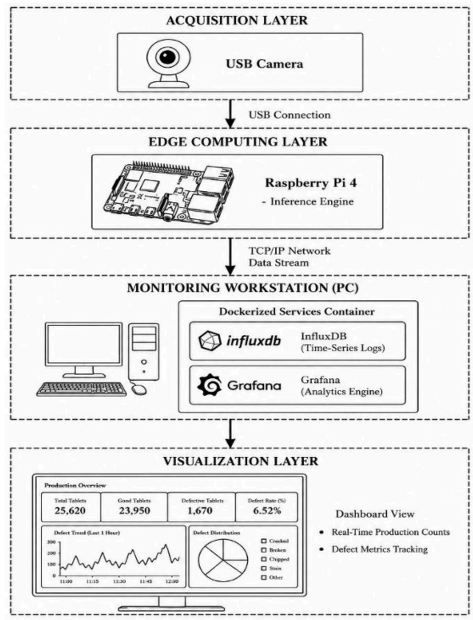

Figure 10. Hardware-to-Software Data flow Pipeline

Model weights were exported to ONNX format to achieve faster CPU inference. The input size for the Pi deployment was reduced to 320 px for Stage 1 and 320 px for Stage 2, with optional frame-skipping to maintain throughput.

5. Conclusion and Future Work

This paper demonstrates an automated, deep learning-based vision framework for real-time, in-line pharmaceutical tablet quality control. By evaluating decoupled and hybrid vision topologies, the system addresses the vulnerabilities of manual batch testing and proprietary OEM equipment. The optimized single-stage YOLO11n-seg backend achieves an outstanding mAP@50 of 96.28% and a Mean Intersection over Union ($mIoU$) of 0.7582. Operating at an ultra-low inference latency of 19.29 ms, the framework delivers a continuous throughput of 42.9 FPS on low-cost edge platforms, vastly outperforming conventional two-stage models like Mask R-CNN. Future work will focus on scaling the pipeline to handle multi-lane conveyor architectures via parallel processing and applying INT8 tensor quantization via TensorRT to compress models further. Additionally, the system will be integrated with high-speed automated pneumatic air-rejection modules to enable fail-safe, continuous removal of defective products during production.

References

[1] N. B. Prajwala, "Defect Detection in Pharma Pills Using Image Processing," Int. J. Eng. Technol., vol. 7, no. 3.3, pp. 102–106, Jun. 2018.

[2] K. A. Pathak, P. Kafle, and A. Vikram, "Deep learning-based defect detection in film-coated tablets using a convolutional neural network," Int. J. Pharm., vol. 670, p. 125220, Jan. 2025.

[3] X. Ma et al., "Application of deep learning convolutional neural networks for internal tablet defect detection: High accuracy, throughput, and adaptability," J. Pharm. Sci., vol. 109, no. 4, pp. 1547–1557, Apr. 2020.

[4] M. Ficzere et al., "Real-time coating thickness measurement and defect recognition of film coated tablets with machine vision and deep learning," Int. J. Pharm., vol. 623, p. 121957, Jun. 2022.

[5] A. Vijayakumar, S. Vairavasundaram, J. A. S. Koilraj, M. Rajappa, K. Kotecha, and A. Kulkarni, "Real-time visual intelligence for defect detection in pharmaceutical packaging," Sci. Rep., vol. 14, p. 18811, Aug. 2024.

[6] A. Diószegi et al., "Automated tablet defect detection and the prediction of disintegration time and crushing strength with deep learning based on tablet surface images," Int. J. Pharm., vol. 667, p. 124896, Dec. 2024.

[7] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Harlow, U.K.: Pearson, 2018.

[8] A. K. Jain, *Fundamentals of Digital Image Processing*. Englewood Cliffs, NJ, USA: Prentice Hall, 1989.

[9] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62–66, 1979.

[10] J. Serra, *Image Analysis and Mathematical Morphology*. London, U.K.: Academic Press, 1982.

[11] L. G. Shapiro and G. C. Stock, *Computer Vision*. Upper Saddle River, NJ, USA: Prentice Hall, 2001.

[12] J. Canny, "A Computational Approach to Edge Detection," IEEE Trans. Pattern Anal. Mach. Intell., vol. 8, no. 6, pp. 679–698, 1986.

[13] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.

[14] D. Forsyth and J. Ponce, *Computer Vision: A Modern Approach*, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2011.

[15] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.